

## ЛАБОРАТОРНА РОБОТА № 9

### 9.1 Тема

Використання регулярних виразів для пошуку та заміни в рядках

### 9.2 Мета роботи

Ознайомитися з основами регулярних виразів у мові програмування Python, вивчити їх синтаксис та навчитися застосовувати регулярні вирази для пошуку, перевірки та заміни фрагментів тексту з використанням модуля *re*.

### х.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- призначення та можливості регулярних виразів у мові програмування Python;
- структуру та синтаксис регулярних виразів;
- основні спеціальні символи та квантифікатори регулярних виразів;
- призначення та принцип роботи основних функцій модуля *re* (*search*, *findall*, *match*, *sub*);
- особливості пошуку та заміни текстових фрагментів із використанням шаблонів.

2) уміти:

- використовувати модуль *re* для пошуку інформації в текстових рядках;
- складати регулярні вирази для розв'язання прикладних задач;
- здійснювати заміну фрагментів тексту з використанням функції *re.sub()*;
- виконувати перевірку коректності рядків за заданими шаблонами (*email*, *телефон*, *IP-адреса* тощо);
- тестувати та налагоджувати програми, що використовують регулярні вирази.

3) набути навичок:

- практичного застосування регулярних виразів для обробки текстових даних;

- аналізу та обробки рядкової інформації з використанням шаблонів;
- побудови ефективних та коректних регулярних виразів;
- самостійного розв’язання типових задач пошуку, перевірки та заміни тексту в Python;
- оформлення результатів роботи у вигляді програмного коду та звіту лабораторної роботи.

#### 9.4 Матеріально-технічне забезпечення

- персональний комп’ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

#### 9.5 Теоретичні відомості

##### Що таке регулярні вирази

Регулярні вирази (англ. regular expressions, regex) – це спеціальні шаблони, які описують множину рядків і дозволяють виконувати пошук, перевірку та заміну текстових даних у програмі.

Вони широко застосовуються для:

***валідації вводу (email, телефон, IP-адреси);***

***пошуку та заміни текстових фрагментів;***

***аналізу великих текстових файлів;***

***обробки логів та даних.***

##### Модуль re у Python

У Python для роботи з регулярними виразами використовується стандартний модуль *re*.

## Основні функції модуля

| Функція                                    | Призначення                                           |
|--------------------------------------------|-------------------------------------------------------|
| <code>re.search(pattern, string)</code>    | Пошук першого входження шаблону у рядку.              |
| <code>re.findall(pattern, string)</code>   | Пошук усіх входжень шаблону у рядку, повертає список. |
| <code>re.match(pattern, string)</code>     | Перевірка, чи починається рядок з шаблону.            |
| <code>re.sub(pattern, repl, string)</code> | Заміна всіх входжень шаблону на заданий рядок.        |
| <code>re.split(pattern, string)</code>     | Розбиття рядка за шаблоном.                           |

## Основні спеціальні символи регулярних виразів

| Символ             | Значення                                               |
|--------------------|--------------------------------------------------------|
| <code>.</code>     | Будь-який символ, окрім символу нового рядка           |
| <code>\d</code>    | Цифра (0–9)                                            |
| <code>\D</code>    | Не цифра                                               |
| <code>\w</code>    | Будь-який символ слова (буква, цифра, <code>_</code> ) |
| <code>\W</code>    | Не символ слова                                        |
| <code>\s</code>    | Пробільний символ (пробіл, табуляція, перенос рядка)   |
| <code>\S</code>    | Не пробільний символ                                   |
| <code>^</code>     | Початок рядка                                          |
| <code>\$</code>    | Кінець рядка                                           |
| <code>*</code>     | 0 або більше повторень попереднього символу            |
| <code>+</code>     | 1 або більше повторень попереднього символу            |
| <code>?</code>     | 0 або 1 повторення попереднього символу                |
| <code>{m,n}</code> | Від m до n повторень попереднього символу              |
| <code>[]</code>    | Множина символів (наприклад, <code>[abc]</code> )      |
| <code>()</code>    | Групування символів або підпатернів                    |

## Складання регулярних виразів

Простий пошук:

```
import re
re.search("Python", "Я вивчаю Python") # знаходить перше входження
```

Пошук всіх входжень:

```
import re
re.findall(r'\d+', "123 abc 45 6") # ['123', '45', '6']
```

Заміна тексту:

```
re.sub(r'\d+', '#', "123 abc 45") # "# abc #"
```

Використання групування:

```
match = re.search(r'(\d+)-(\d+)', '12-34')
match.groups() # ('12', '34')
```

## 5. Особливості роботи в Python

1) Використовуються raw strings (r'') для регулярних виразів, щоб не потрібно було екранувати символи \.

2) Регулярні вирази можна комбінувати та ускладнювати для пошуку складних шаблонів.

3) Важливо розуміти різницю між функціями:

- search – повертає перший збіг;
- findall – повертає список усіх збігів;
- match – перевіряє тільки початок рядка.

## 9.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.
- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

## 9.7 Загальне завдання

Написати програму, яка:

- вводить початкові дані (з файлу або з клавіатури);
- формує результат з використанням регулярних виразів;
- виводить результат на екран або зберігає у файл.

## 9.8 Індивідуальні завдання

- 1) Реалізувати програму для пошуку та заміни всіх цифр у рядку на символ #.
- 2) Написати програму для пошуку електронних адрес у тексті.
- 3) Створити програму, яка видаляє всі зайві пробіли з рядка.
- 4) Написати програму для пошуку всіх слів, що починаються з великої літери.
- 5) Реалізувати програму для перевірки, чи є рядок коректним телефонним номером.
- 6) Створити програму для видалення всіх небажаних символів з тексту.
- 7) Написати програму для пошуку всіх email-адрес у тексті.
- 8) Реалізувати програму для заміни всіх дат у форматі dd-mm-yyyy на yyyy-mm-dd.
- 9) Створити програму для перевірки, чи є рядок валідною IP-адресою.
- 10) Написати програму для заміни всіх пробілів у рядку на символ підкреслення \_.

### Нульовий варіант (приклад реалізації)

Написати програму, яка знаходить у рядку всі числа та замінює їх на символ #.

#### Програмний код

```
# Лабораторна робота №9
# Імпортуємо модуль для роботи з регулярними виразами
import re

# Вхідний текст
text = "У цьому рядку є числа: 123, 45 і 6789."

# Пошук всіх чисел у тексті
# \d+ означає "одна або більше цифр"
numbers_found = re.findall(r'\d+', text)
print("Знайдені числа:", numbers_found)

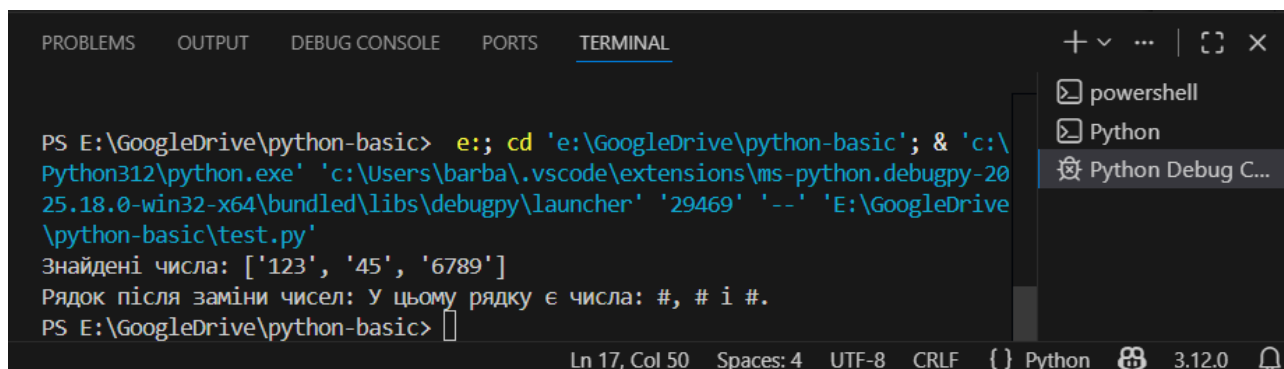
# Замінюємо всі числа на символ #
# \d+ означає "одна або більше цифр"
```

```
# re.sub(pattern, repl, string) замінює всі входження pattern на repl у рядку
string
text_modified = re.sub(r'\d+', '#', text)
print("Рядок після заміни чисел:", text_modified)
```

### Пояснення коду

- import re – підключаємо стандартний модуль для роботи з регулярними виразами.
- re.findall(r'\d+', text) – шукає всі послідовності цифр у рядку text і повертає їх у вигляді списку.
- re.sub(r'\d+', '#', text) – замінює всі знайдені числа на символ #.
- Використання r" (raw string) дозволяє не екранувати спецсимволи в регулярному виразі.
- Програма виводить результати у зручному для перевірки вигляді.

### Результат роботи



```
PS E:\GoogleDrive\python-basic> e;; cd 'e:\GoogleDrive\python-basic'; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '29469' '--' 'E:\GoogleDrive\python-basic\test.py'
Знайдені числа: ['123', '45', '6789']
Рядок після заміни чисел: У цьому рядку є числа: #, # і #.
PS E:\GoogleDrive\python-basic>
```

## 9.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

### 9.10 Контрольні питання

- 1) Як працює функція ***re.search()*** у Python?
- 2) Які спеціальні символи використовуються в регулярних виразах?
- 3) Як за допомогою регулярних виразів знайти всі цифри в рядку?
- 4) Для чого використовується функція ***re.sub()***?
- 5) Як перевірити наявність певної підстроки в рядку за допомогою регулярних виразів?

### 9.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.  
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>